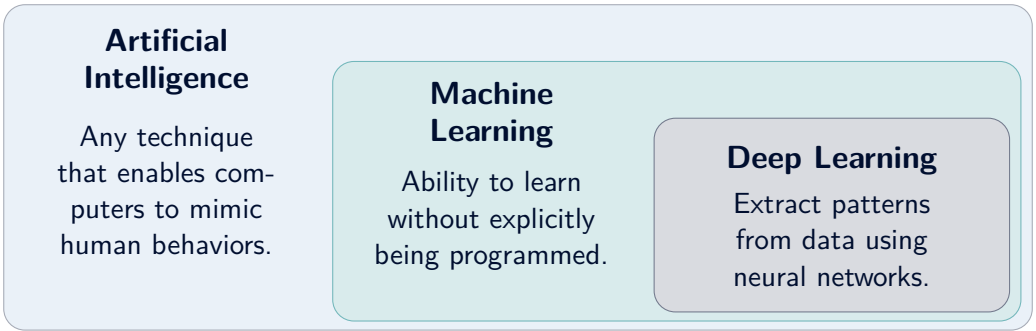
Deep learning foundations

Lecture DL-1: Overview, MLPs, and training

Xingche Guo

STAT 5095 University of Connecticut · Fall 2026

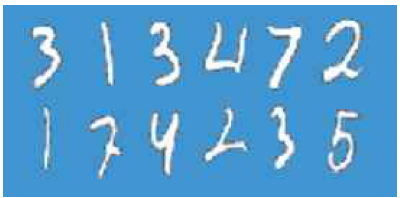
What is deep learning?



Deep learning teaches computers how to **learn a task** directly from **raw data**.

Adapted from MIT 6.S191, Introduction to Deep Learning, Lecture 1, slide 7. Alexander Amini and Ava Amini. introtodeeplearning.com

Recognition and generation



Recognition
Image $x \longrightarrow p_\theta(y \mid x)$
Which digit is this?

Generation
Randomness $z \longrightarrow$ new image
What could another digit look like?

The neural network is a flexible function. The objective determines its job.

Images: supplied MIT 6.S191, Alexander Amini and Ava Amini, 5 January 2026. introtodeeplearning.com.

Generating images





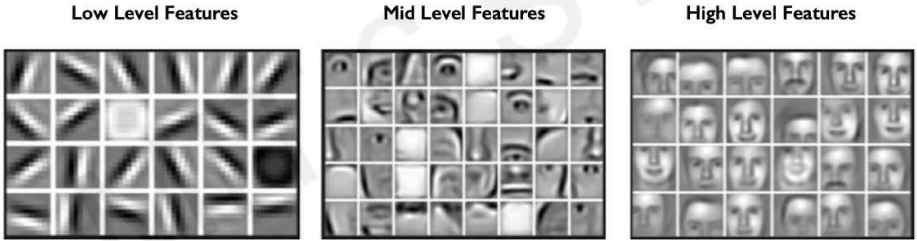
Can hand-engineered rules capture the diversity of real faces?

Facial shape, expression, skin texture, and lighting vary together. Writing explicit rules for all these variations is difficult, time consuming, and not scalable in practice.

Instead, learn the distribution of faces from data.

Images: supplied MIT 6.S191, Alexander Amini and Ava Amini, 5 January 2026. introtodeeplearning.com.

Feature hierarchies



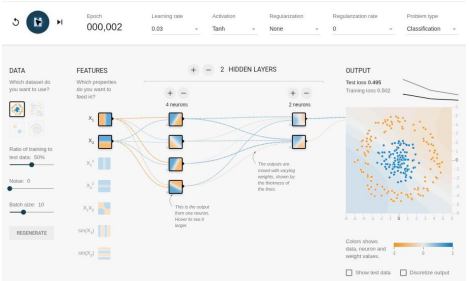
Deep learning learns a hierarchy of features from data.

Each layer builds on the previous layer's representation: simple patterns are combined into more complex features.

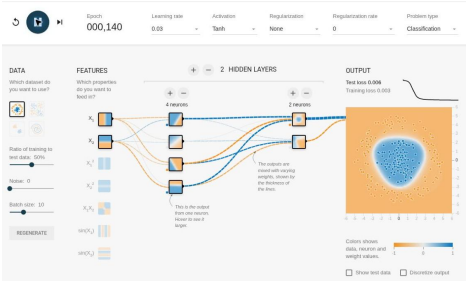
Image: supplied MIT 6.S191, slide 18. Alexander Amini and Ava Amini, 5 January 2026. introtodeeplearning.com.

A minimal deep learning demo

Task: Use a point's two coordinates to predict its color.



Early in training



After learning

▸ Run the live demo

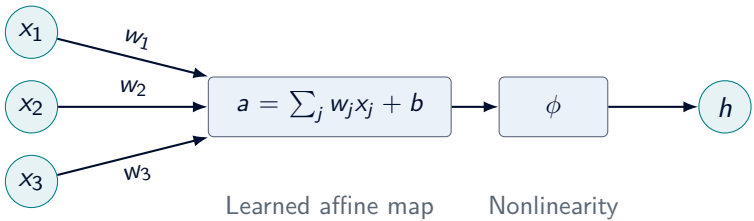
Try it: Reset, press Play, and pause.

What changes as the network learns to separate the colors?

Actual screenshots: TensorFlow Playground, Daniel Smilkov and Shan Carter; Apache 2.0. Results depend on initialization and training duration.

One neuron (Perceptron)

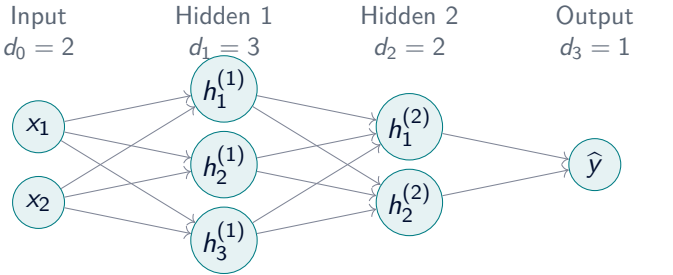
$$a = \sum_{j=1}^{d} w_j x_j + b = w^\top x + b, \qquad h = \phi(a).$$



- Weights w and bias b are learned parameters.
- The activation ϕ introduces a nonlinear transformation.

Multilayer perceptron (MLP)

$$a^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \qquad h^{(l)} = \phi_l(a^{(l)}).$$



$$h^{(0)} = x, \qquad \widehat{y} = f_\theta(x) = h^{(3)}.$$

An MLP is a composition of fully connected layers and activations.

MLP for all subjects

For n subjects, each with d_0 input features, define

$$X = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix} \in \mathbb{R}^{d_0 \times n}, \qquad H^{(0)} = X.$$

The entire dataset passes through layer l simultaneously:

$$\underbrace{H^{(l)}}_{d_l \times n} = \phi_l \left(\underbrace{W^{(l)}}_{d_l \times d_{l-1}} \underbrace{H^{(l-1)}}_{d_{l-1} \times n} + \underbrace{b^{(l)} \mathbf{1}_n^\top}_{d_l \times n} \right).$$

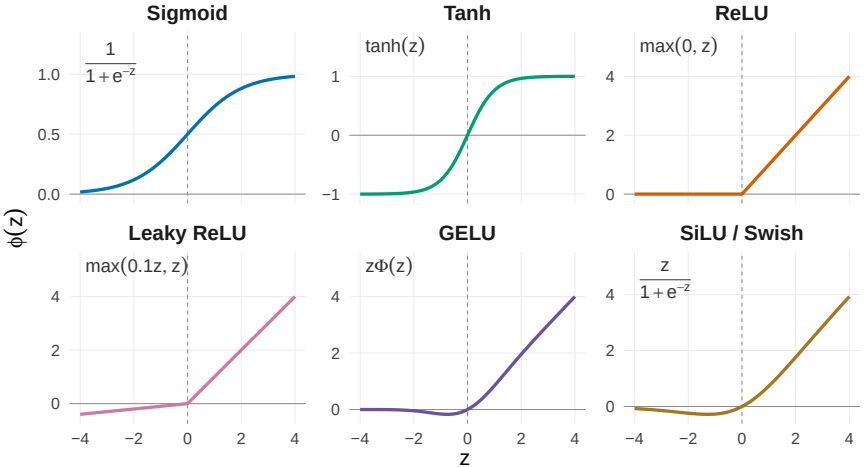
- Column i of $H^{(l)}$ is subject i's representation after layer l.
- $b^{(l)} \mathbf{1}_n^\top$ repeats the same bias vector across all n subjects.

After the final layer:

$$\widehat{Y} = H^{(L)} \in \mathbb{R}^{d_L \times n}.$$

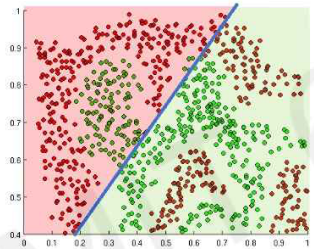
Every subject is processed using the same weights and biases.

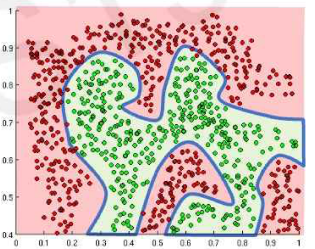
Activation functions



Common x range; y scales vary by panel. Phi denotes the standard normal CDF.

Why nonlinearity?





With identity activations, an MLP with one hidden layer reduces to

$$f(x) = W_2(W_1 x + b_1) + b_2 = \underbrace{(W_2 W_1)}_{W_{\text{eff}}} x + \underbrace{(W_2 b_1 + b_2)}_{b_{\text{eff}}}.$$

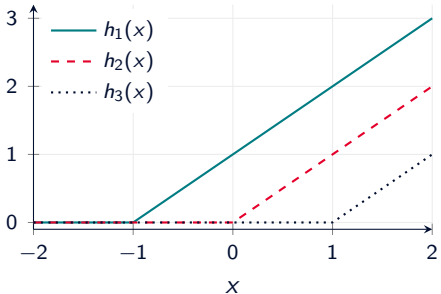
- Stacking affine layers still gives a single affine map.
- Nonlinear activations allow the network to learn nonlinear features.

Would adding more layers help if every activation were the identity?

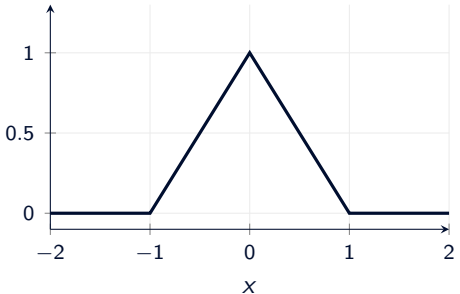
A curve from simple units

$$f(x) = \mathrm{ReLU}(x+1) - 2\,\mathrm{ReLU}(x) + \mathrm{ReLU}(x-1).$$

Hidden features



Learned combination



The output combines learned nonlinear features.

Train a neural network

Given training data $\{(x_i, y_i)\}_{i=1}^n$, the network predicts

$$\widehat{y}_i = f_\theta(x_i), \qquad \theta = \{W_\ell, b_\ell\}_{\ell=1}^L,$$

where θ contains all trainable weights and biases.

Training means minimizing the empirical loss:

$$\min_\theta \; \mathcal{L}_n(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_\theta(x_i)) \,.$$

- $\ell(y_i, \widehat{y}_i)$ measures the error for one observation.
- $\mathcal{L}_n(\theta)$ averages these errors over the training data.
- We adjust θ so that the network's predictions better match the observed targets.

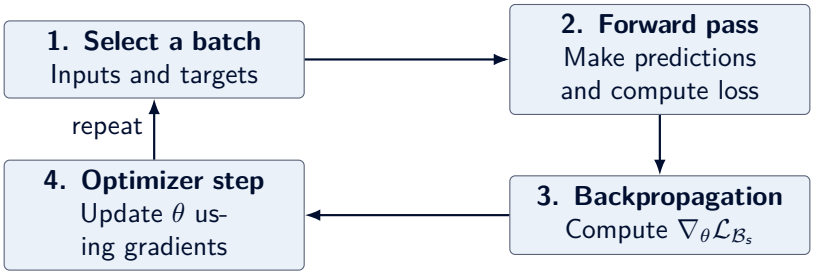
Find the proper loss

Let a denote the network's final score, before the output activation.

Task	Network output	Per-example loss
Regression	$\widehat{y} = a$	$\frac{1}{2}\|\widehat{y} - y\|_2^2$
Binary classification	$p = \text{sigmoid}(a)$	$-y \log p - (1-y)\log(1-p)$
One of K classes	$\mathbf{p} = \text{softmax}(\mathbf{a})$	$-\sum_{k=1}^{K} y_k \log p_k$

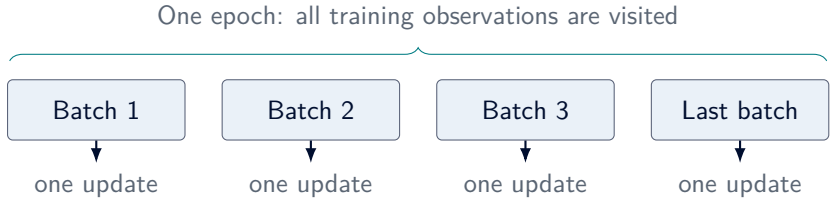
- **Regression:** squared error compares predicted and observed values.
- **Classification:** cross-entropy rewards assigning high probability to the correct class.

Four training operations



Backpropagation computes gradients. The optimizer uses them to update parameters.

1. Select a batch



- **Batch:** a group of training examples processed together. At step s, select indices $\mathcal{B}_s$ with batch size $B_s = |\mathcal{B}_s|$.
- **Step:** one training iteration: forward pass $\rightarrow$ backpropagation $\rightarrow$ parameter update.
- **Epoch:** one complete pass through the training set, usually using several batches.

Example: $n = 1000$ training examples, batch size $B = 128$:

$$\underbrace{7 \times 128 + 104}_{\text{all 1000 examples}} \quad \Longrightarrow \quad \text{8 updates per epoch.}$$

2. Forward pass

Using the current parameters θ_s, pass the batch inputs through the network to compute predictions and a loss.

1. Compute predictions. For an MLP with one hidden layer, each $i \in \mathcal{B}_s$ follows

$$h_i = \phi(W_1 x_i + b_1), \qquad \widehat{y}_i = \psi(W_2 h_i + b_2).$$

Here, ϕ is the hidden activation and ψ is the output activation chosen for the task.

2. Compute the average loss over the batch.

$$\boxed{\mathcal{L}_{\mathcal{B}_s}(\theta_s) = \frac{1}{B_s} \sum_{i \in \mathcal{B}_s} \ell(y_i, \widehat{y}_i)\,.}$$

3. Backpropagation

The forward pass computes the mini-batch loss:

$$\mathcal{L}_{\mathcal{B}_s}(\theta) = \frac{1}{B_s} \sum_{i \in \mathcal{B}_s} \ell(y_i, f_\theta(x_i)) \,.$$

Backpropagation computes its gradient with respect to all weights and biases:

$$\boxed{\widehat{g}_s = \nabla_\theta \mathcal{L}_{\mathcal{B}_s}(\theta)|_{\theta=\theta_s} \,.}$$

- Work backward from the loss through the network.
- Apply the **chain rule** through successive layers.
- Each gradient component measures how the loss changes locally with one parameter.

Matrix chain rule formula

For a scalar loss $J(Y)$, the matrix gradient $G_Y := \partial J / \partial Y$ is defined by

$$dJ = \sum_{u,v} \frac{\partial J}{\partial Y_{uv}} dY_{uv} = \sum_{u,v} (G_Y)_{uv}\, dY_{uv} = \operatorname{tr}(G_Y^\top dY).$$

Suppose $Y = AXB + C$, with A, B, and C fixed. Then

$$dY = A\,(dX)\,B.$$

Substitute into dJ and use the cyclic property of the trace:

$$\begin{aligned} dJ &= \operatorname{tr}\Big(G_Y^\top A\,(dX)\,B\Big) \\ &= \operatorname{tr}\Big(BG_Y^\top A\,dX\Big). \end{aligned}$$

Comparing with $dJ = \operatorname{tr}(G_X^\top dX)$ gives

$$\boxed{\frac{\partial J}{\partial X} = A^\top \frac{\partial J}{\partial Y} B^\top.}$$

Backpropagation algorithm

For a batch $\mathcal{B}$ of size B, consider each observation $i \in \mathcal{B}$. Let $h_i^{(0)} = x_i$.

Forward pass:

$$\begin{aligned} z_i^{(k)} &= W^{(k)} h_i^{(k-1)} + b^{(k)}, \quad && k = 1, \dots, L, \\ h_i^{(k)} &= \phi_k(z_i^{(k)}), && k = 1, \dots, L-1, \\ \widehat{y}_i &= \psi(z_i^{(L)}). \end{aligned}$$

Per-observation loss and average batch loss:

$$\ell_i = \ell(y_i, \widehat{y}_i), \qquad L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell_i(\theta).$$

Our goal: compute the gradient for every layer.

$$\boxed{\frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial W^{(k)}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \frac{\partial \ell_i}{\partial W^{(k)}}, \qquad \frac{\partial \mathcal{L}_{\mathcal{B}}}{\partial b^{(k)}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \frac{\partial \ell_i}{\partial b^{(k)}}.}$$

Backpropagation algorithm

Recall the affine transformation:

$$z_i^{(k)} = W^{(k)} h_i^{(k-1)} + b^{(k)}.$$

Weight gradient: combine the backward gradient with the incoming activation.

$$\boxed{\frac{\partial \ell_i}{\partial W^{(k)}} = \frac{\partial \ell_i}{\partial z_i^{(k)}} (h_i^{(k-1)})^\top.}$$

Bias gradient: the bias enters additively, so

$$\boxed{\frac{\partial \ell_i}{\partial b^{(k)}} = \frac{\partial \ell_i}{\partial z_i^{(k)}}.}$$

There, it remains to compute

$$\frac{\partial \ell_i}{\partial z_i^{(k)}}, \qquad k = L, L-1, \ldots, 1.$$

Backpropagation algorithm

Start at the output layer: For an elementwise output activation $\widehat{y}_i = \psi(z_i^{(L)})$,

$$\boxed{\frac{\partial \ell_i}{\partial z_i^{(L)}} = \frac{\partial \ell_i}{\partial \widehat{y}_i} \odot \psi'(z_i^{(L)}).}$$

Apply the chain rule backward ($k = L-1, \ldots, 1$): For an elementwise output activation $\widehat{h}_i^{(k)} = \phi_k(z_i^{(k)})$,

$$\frac{\partial \ell_i}{\partial z_i^{(k)}} = \frac{\partial \ell_i}{\partial h_i^{(k)}} \odot \phi_k'(z_i^{(k)}).$$

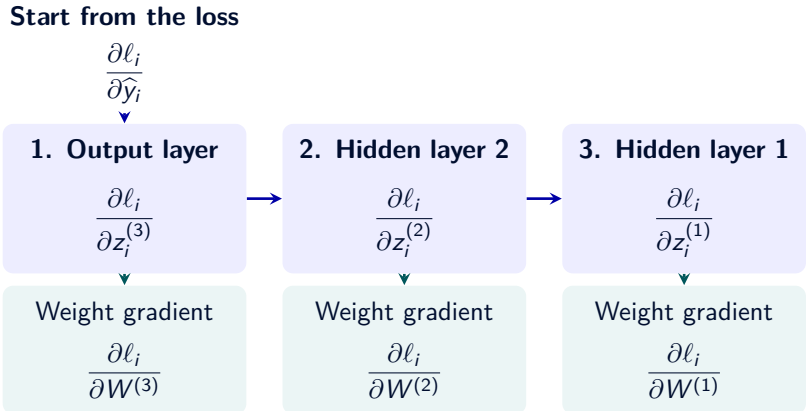
Because $z_i^{(k+1)} = W^{(k+1)} h_i^{(k)} + b^{(k+1)}$,

$$\frac{\partial \ell_i}{\partial h_i^{(k)}} = \left(W^{(k+1)}\right)^\top \frac{\partial \ell_i}{\partial z_i^{(k+1)}}.$$

Therefore

$$\boxed{\frac{\partial \ell_i}{\partial z_i^{(k)}} = \left[\left(W^{(k+1)}\right)^\top \frac{\partial \ell_i}{\partial z_i^{(k+1)}}\right] \odot \phi_k'(z_i^{(k)}).}$$

Computation order



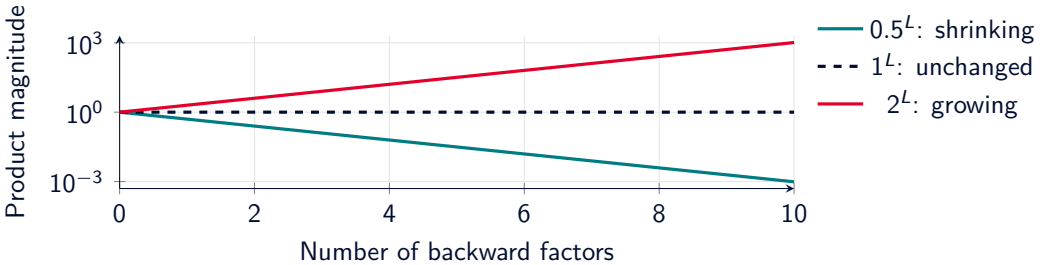
At each layer, obtain its weight gradient and propagate backward.

Gradients through depth

Recall the backward recursion:

$$\frac{\partial \ell_i}{\partial z_i^{(k)}} = \left[\left(W^{(k+1)}\right)^\top \frac{\partial \ell_i}{\partial z_i^{(k+1)}} \right] \odot \phi_k'(z_i^{(k)}).$$

Each layer introduces another multiplication by **weights and activation derivatives**.



- **Vanishing gradients:** early layers receive weak learning signals, slowing training.
- **Exploding gradients:** large gradients can cause oversized updates and numerical

Why is ReLU popular?

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

- **No saturation on the positive side.** For an active unit, the activation derivative is 1, so this factor does not shrink the backward gradient.
- **Simple and inexpensive.** The forward pass takes a maximum; the backward pass applies a 0/1 mask.

For comparison, the sigmoid derivative satisfies

$$\sigma(z) = \frac{e^z}{1 + e^z}, \qquad \sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4},$$

and approaches zero for large-magnitude inputs.

Glorot, Bordes & Bengio (2011).

4. Optimization

Gradient descent takes a step opposite to a loss gradient.

In **mini-batch stochastic gradient descent (SGD)**, use the batch gradient $\widehat{g}_s$ computed by backpropagation:

$$\boxed{\theta_{s+1} = \theta_s - \eta_s \widehat{g}_s.}$$

- θ_s: current weights and biases.
- $\eta_s > 0$: the **learning rate**, controlling the step size.
- Different batches give different gradient estimates, so the training trajectory can fluctuate.

Does a larger learning rate always lead to faster improvement?

▸ Demo: explore the learning rate

Choosing the learning rate

Learning rate	What can happen?
Too small	Tiny updates: slow progress before reaching a useful solution.
Too large	Updates overshoot: the loss may oscillate, increase, or diverge.

How can we choose a suitable learning rate?

1. **Try several values on a logarithmic scale.**
   For example, $10^{-4}, 10^{-3}, 10^{-2}$. Use short runs to screen for stable progress, then refine the range and compare validation performance.

2. **Use an adaptive optimizer, such as Adam.**
   It uses past gradients to adjust the update scale for each parameter. The base learning rate still needs tuning.

Kingma & Ba: Adam.

Optimizers beyond SGD

All optimizers use gradients from backpropagation. They differ in **how those gradients determine the update**.

Optimizer	Main idea
SGD with momentum	Combine current and past gradients to smooth updates and accelerate consistent movement.
Adagrad	Scale updates using accumulated squared gradients; effective learning rates decrease over time.
RMSprop	Scale updates using a moving average of squared gradients, emphasizing recent information.
Adam	Combine moving averages of gradients and squared gradients to adapt updates for each parameter.
AdamW	Use Adam with **decoupled weight decay**: shrink weights separately from the adaptive update.

Momentum, Adagrad, RMSprop; Adam: Kingma & Ba; AdamW: Loshchilov & Hutter.

Momentum SGD and Adagrad

At iteration $s = 0, 1, \ldots$, compute the mini-batch gradient:

$$g_s = \nabla_\theta \mathcal{L}_{\mathcal{B}_s}(\theta_s).$$

1. Momentum SGD: accumulate gradients. Initialize $v_0 = 0$.

$$v_{s+1} = \mu v_s + (1 - \mu) g_s,$$
$$\theta_{s+1} = \theta_s - \eta_s v_{s+1}.$$

The momentum coefficient $\mu \in [0, 1)$ controls how much past gradients contribute; $\mu = 0$ recovers ordinary SGD.

2. Adagrad: adapt the scale for each parameter. Initialize $r_0 = 0$.

$$r_{s+1} = r_s + g_s \odot g_s,$$
$$\theta_{s+1} = \theta_s - \eta_s \frac{g_s}{\sqrt{r_{s+1}} + \epsilon}.$$

Larger accumulated squared gradients give smaller effective learning rates.

Stanford CS231n: momentum and Adagrad.

Adam

Use the same mini-batch gradient g_s. Initialize $m_0 = v_0 = 0$.

1. Track moving averages of gradients and squared gradients.

$$m_{s+1} = \beta_1 m_s + (1 - \beta_1) g_s,$$
$$v_{s+1} = \beta_2 v_s + (1 - \beta_2)(g_s \odot g_s).$$

2. Correct for initialization at zero.

$$\widehat{m}_{s+1} = \frac{m_{s+1}}{1 - \beta_1^{s+1}}, \qquad \widehat{v}_{s+1} = \frac{v_{s+1}}{1 - \beta_2^{s+1}}.$$

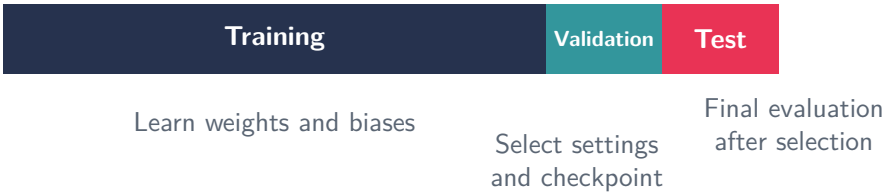
3. Update the parameters.

$$\boxed{\theta_{s+1} = \theta_s - \eta_s \frac{\widehat{m}_{s+1}}{\sqrt{\widehat{v}_{s+1}} + \epsilon}.}$$

- $\widehat{m}$: smoothed gradient; $\widehat{v}$: scale from recent squared gradients.
- Common settings: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$; still tune η_s.

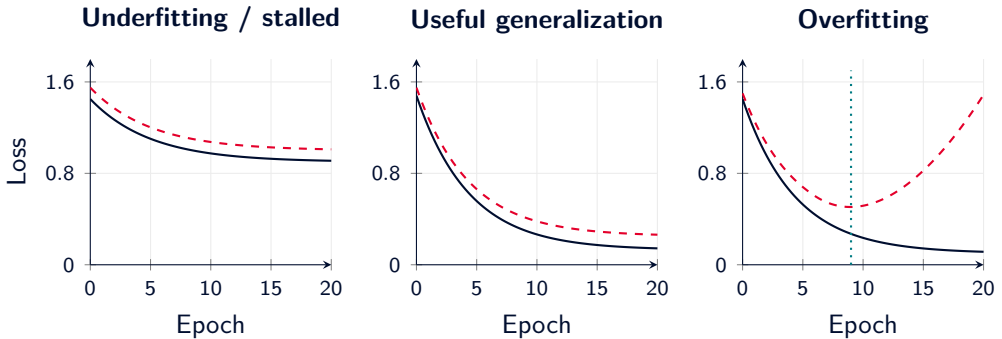
Kingma & Ba (2015): Adam.

Fit, validate, then test



- Use **training data** to learn the network parameters (weights and biases).
- Use **validation data** to tune hyperparameters, compare architectures, and select a checkpoint.
- Use **test data** to evaluate the final selected model on unseen observations.

Read the learning curves



- High training and validation loss: inspect fit, optimization, and data.
- Low training loss, rising validation loss: investigate overfitting.
- Unstable or nonfinite loss: inspect scale, rate, and computation.

Illustrative curves, not experimental measurements.

How to reduce overfitting

- **Dropout:** Randomly set some hidden activations to zero during training, reducing reliance on particular neurons. Disable dropout during validation and testing.
- **Early stopping:** Stop when validation loss stops improving for several epochs; restore the checkpoint with the lowest validation loss.
- **Weight penalties (L_1 or L_2):** Add a penalty to the training loss:

$$J(\theta) = \mathcal{L}_{\mathcal{B}}(\theta) + \lambda R(\theta), \qquad R(\theta) = \|\theta\|_1 \text{ or } \|\theta\|_2^2.$$

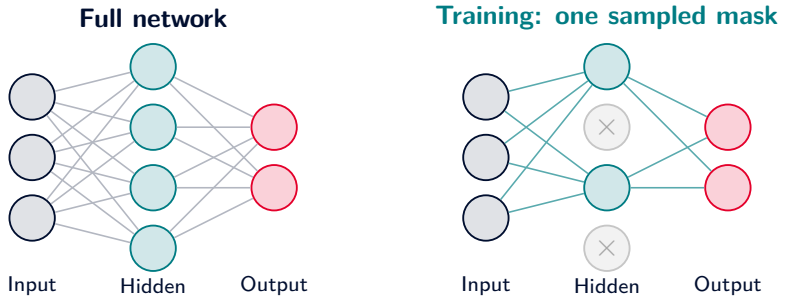
  L_1 encourages sparsity; L_2 discourages large weights.
- **More data or data augmentation:** Collect more representative examples or create valid variations of training examples, such as label-preserving image transformations.

Choose regularization settings using validation performance.

Stanford CS231n: regularization; TensorFlow: overfitting.

Dropout

During training, randomly set some hidden activations to zero.



- The mask changes across training examples and forward passes.
- Neurons are **temporarily masked**, not permanently removed.
- This discourages reliance on a particular combination of neurons.

Srivastava et al. (2014): Dropout.

Dropout

For observation i, let $h_i^{(k)} = \phi_k(z_i^{(k)})$. Let $p \in [0, 1)$ be the **probability of dropping an activation**.

Training: sample a mask and rescale retained activations.

$$r_{ij}^{(k)} \overset{\text{ind}}{\sim} \text{Bernoulli}(1-p), \qquad \boxed{\widetilde{h}_i^{(k)} = \frac{r_i^{(k)} \odot h_i^{(k)}}{1-p}.}$$

Pass the masked activations to the next layer:

$$z_i^{(k+1)} = W^{(k+1)} \widetilde{h}_i^{(k)} + b^{(k+1)}.$$

Why divide by $1-p$? To preserve the expected activation: $\mathbb{E}\left[\widetilde{h}_i^{(k)} \,\middle|\, h_i^{(k)}\right] = h_i^{(k)}$.

Validation and testing: disable dropout.

$$\boxed{\widetilde{h}_i^{(k)} = h_i^{(k)}.}$$

No random masking or additional rescaling is needed.

PyTorch: inverted dropout.

Dropout in mini-batch training

At each training iteration:

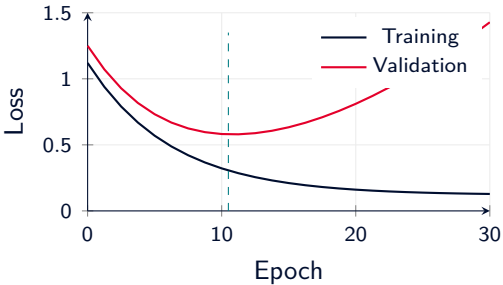
1. Select a mini-batch $\mathcal{B}_s$ and sample fresh dropout masks **for each observation**.
2. Run the forward pass with these masks, then backpropagate using **the same masks**.
3. Compute the gradients and apply one optimizer update.

Need to sample fresh dropout masks for each new mini-batch.

	Training	**Validation / test**
Dropout	Random masking and scaling	Disabled
Parameters	Updated using gradients	Held fixed

PyTorch: Dropout; training and evaluation modes.

Early stopping



1. Monitor a validation criterion.
2. Save the best checkpoint.
3. Stop after a patience window without sufficient improvement.
4. Restore the best checkpoint.

Initializing weights and biases

For layer k, $z_i^{(k)} = W^{(k)} h_i^{(k-1)} + b^{(k)}, \qquad W^{(k)} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}.$

Two common choices: sample weight entries independently.

$$\text{Xavier / Glorot:} \qquad W_{uv}^{(k)} \overset{\text{iid}}{\sim} \mathcal{N}\left(0, \frac{2}{d_{\text{in}} + d_{\text{out}}}\right),$$

$$\text{He / ReLU:} \qquad W_{uv}^{(k)} \overset{\text{iid}}{\sim} \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right).$$

- Random weights break symmetry between hidden neurons.
- Biases can usually be initialized to zero: $b^{(k)} = 0$.

Choose the weight scale to keep activations and gradients well behaved.

Glorot & Bengio (2010); He et al. (2015).